UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

FACULTAD DE INGENIERIA

SISTEMA DE DRONES

DOCUMENTACIÓN

SERGIO RAÚL QUEZEL YOL
ESTUDIANTE DE CIENCIAS Y SISTEMAS

GUATEMALA, ABRIL DE 2026

DOCUMENTACIÓN DEL PROYECTO

Sistema de Simulación de Drones

INTRODUCCIÓN

El presente proyecto consiste en el desarrollo de un sistema capaz de simular el funcionamiento de múltiples drones para la generación de mensajes visuales, a partir de configuraciones definidas en un archivo XML.

El sistema permite:

1. Interpretar configuraciones de drones y sistemas
2. Procesar mensajes con instrucciones específicas
3. Simular el comportamiento de los drones en el tiempo

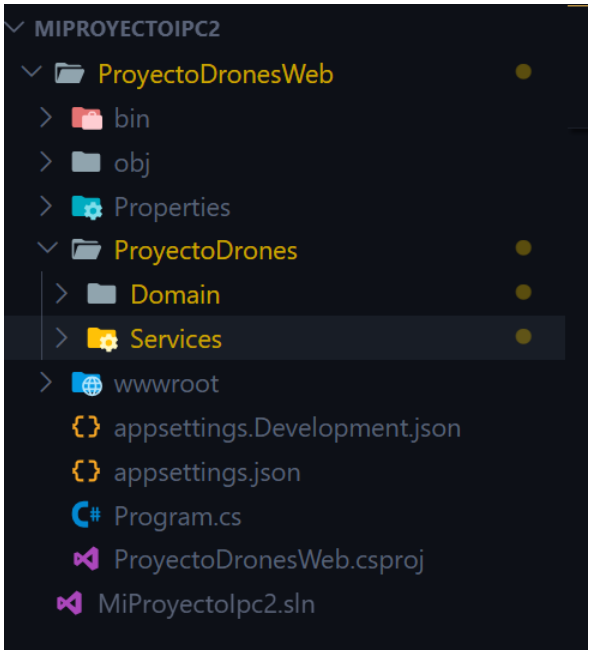
OBJETIVO GENERAL

Desarrollar un sistema que permita la simulación eficiente de drones para la construcción de mensajes, garantizando:

1. Correcta interpretación de datos XML
2. Ejecución de instrucciones
3. Visualización del proceso mediante gráficos
4. Gestión de entidades del sistema

ARQUITECTURA DEL SISTEMA

El sistema sigue una arquitectura modular:



Sistema de Simulación de Drones

El proyecto está diseñado como un sistema completo que integra procesamiento de datos, simulación lógica y visualización gráfica. Su estructura está organizada en módulos claramente definidos, donde cada archivo cumple una responsabilidad específica dentro del flujo general del sistema. A continuación, se describe el funcionamiento desde una perspectiva estructural, explicando el rol de cada componente.

Capa Domain:

Desde de la carpeta Domain se encuentran las clases que representan las entidades del sistema. Estas clases no contienen lógica compleja, sino que definen la forma en la que los datos son almacenados y organizados.

La clase Drone representa un dron individual dentro del sistema. Su propósito es encapsular el nombre del dron, el cual funciona como identificador único. Aunque su estructura es simple, es fundamental porque los drones son las unidades básicas que ejecutan las instrucciones.

```
6 references | You, 5 days ago | 1 author (You)
public class Drone
{
    3 references
    public string Nombre;
    1 reference
    public int AlturaActual;
    1 reference
    public bool LuzEncendida;

    1 reference
    public Drone(string nombre)
    {
        Nombre = nombre;
        AlturaActual = 0;
        LuzEncendida = false;
    }
}
```

La clase SistemaDrones modela un sistema completo de drones. Este archivo es uno de los más importantes del proyecto, ya que contiene la representación de cómo los drones pueden generar caracteres en distintas alturas. Internamente, maneja una estructura matricial donde cada fila corresponde a un dron y cada columna a una altura. En esta matriz se almacenan caracteres, lo que permite que, al combinar las posiciones de varios drones, se pueda formar un mensaje visual.

```
11 references | You, 5 days ago | 1 author (You)
public class SistemaDrones
{
    6 references
    public string Nombre;
    2 references
    public int AlturaMaxima;
    4 references
    public int CantidadDrones;

    4 references
    public char[,] Mapa;

    12 references
    public string[] NombresDrones;

    1 reference
    public SistemaDrones(string nombre, int cantidad, int alturaMax)
    {
        Nombre = nombre;
        CantidadDrones = cantidad;
        AlturaMaxima = alturaMax;

        Mapa = new char[cantidad, alturaMax + 1];
        NombresDrones = new string[cantidad];
    }

    3 references
    public int ObtenerIndiceDron(string nombre)
    {
        for (int i = 0; i < NombresDrones.Length; i++)
        {
            if (NombresDrones[i] == nombre)
                return i;
        }
        return -1;
    }
```

La clase Mensaje encapsula la definición de un mensaje que se desea construir. Contiene el nombre del mensaje, el sistema de drones asociado y una colección de instrucciones. Estas instrucciones determinan cómo se deben posicionar los drones a lo largo del tiempo.

```
public class Mensaje
{
    4 references
    public string Nombre;
    3 references
    public string Sistema;
    4 references
    public Lista<Instruccion> Instrucciones;

    1 reference
    public Mensaje(string nombre, string sistema)
    {
        Nombre = nombre;
        Sistema = sistema;
        Instrucciones = new Lista<Instruccion>();
    }
}
```

La clase Instrucción representa una acción específica que un dron debe ejecutar. Cada instrucción contiene el nombre del dron y la altura a la que debe desplazarse. Esta clase es utilizada directamente por el motor de simulación.

Adicionalmente, el proyecto utiliza estructuras de datos personalizadas como listas enlazadas. Estas estructuras permiten almacenar colecciones de datos sin depender de las listas nativas del lenguaje, lo cual demuestra control sobre la gestión de memoria y recorridos secuenciales.

Capa Services: Lógica del sistema

La carpeta Services contiene la lógica principal del sistema. Aquí es donde los datos se procesan, transforman y utilizan para generar resultados.

El archivo XMLService es responsable de interpretar el archivo XML de entrada. Su función es leer el documento, recorrer sus nodos y convertirlos en objetos del dominio. Este servicio construye las listas de drones, sistemas y mensajes. Es importante notar que el XML define completamente el estado inicial del sistema, por lo que este archivo actúa como el punto de entrada de los datos.

El archivo SimulacionService implementa el núcleo del proyecto. Aquí se ejecuta la simulación de los drones. A partir de un mensaje y un sistema de drones, este servicio recorre las instrucciones y genera una línea de tiempo que representa las acciones de cada dron. Además, reconstruye el mensaje final interpretando las posiciones de los drones en cada instante. Este componente traduce instrucciones abstractas en un resultado concreto.

El archivo OutputService se encarga de generar el archivo XML de salida. Una vez finalizada la simulación, este servicio toma el mensaje reconstruido y la información relevante del proceso para estructurar un

nuevo XML. Este archivo sirve como evidencia del resultado del sistema.

El archivo GraphvizService tiene como objetivo generar representaciones visuales del sistema. A partir de los datos procesados, construye archivos en formato .dot, que posteriormente son convertidos a imágenes .png. Estas imágenes permiten visualizar tanto la estructura del sistema de drones como la secuencia de instrucciones ejecutadas, facilitando la comprensión del proceso.

Capa de Aplicación

El archivo Program.cs actúa como punto de entrada del sistema web. Implementa una API minimalista utilizando .NET, donde se definen distintos endpoints que permiten interactuar con el sistema.

A través de este archivo, el usuario puede subir un archivo XML, solicitar el procesamiento de un mensaje específico, obtener listados de drones, mensajes y sistemas, así como generar visualizaciones. Este archivo coordina la interacción entre los servicios y expone la funcionalidad al exterior.

Además, aquí se maneja cierta lógica auxiliar, como el cálculo del tiempo óptimo de la simulación, que se obtiene analizando la línea de tiempo generada.

Frontend:

El frontend está construido como una interfaz web sencilla en HTML, CSS y JavaScript. Su objetivo es facilitar la interacción del usuario con el sistema sin necesidad de utilizar la consola.

Desde esta interfaz, el usuario puede cargar un archivo XML, seleccionar un mensaje, ejecutar la simulación y visualizar los resultados. También permite gestionar drones y explorar sistemas. La comunicación con el backend se realiza mediante llamadas a los endpoints definidos en la API.

Las imágenes generadas por Graphviz se integran directamente en la interfaz, lo que permite observar gráficamente tanto la estructura del sistema como la ejecución de las instrucciones.

Flujo general del sistema

El funcionamiento del sistema sigue una secuencia clara. Primero, el usuario carga un archivo XML que define drones, sistemas y mensajes. Luego, el sistema procesa esta información y la almacena en estructuras internas. Posteriormente, el usuario selecciona un mensaje, lo que desencadena la simulación. Durante esta etapa, se ejecutan las instrucciones, se construye una línea de tiempo y se reconstruye el mensaje final.

Finalmente, el sistema genera un archivo XML de salida y dos representaciones gráficas: una del sistema de drones y otra del proceso de ejecución. Estos resultados se muestran en la interfaz web.

Consideraciones de diseño

El sistema fue diseñado con una separación clara entre modelo, lógica y presentación. Esta organización facilita el mantenimiento, la escalabilidad y la comprensión del código.

El uso de XML como entrada permite definir configuraciones complejas de forma estructurada sin modificar el código fuente. Por otro parte, el uso de Graphviz aporta una capa visual que mejora significativamente la interpretación del sistema.

El uso de estructuras de datos propias refuerza el aprendizaje en el manejo de memoria y recorridos, lo cual es clave en la formación académica.

Conclusión

El proyecto representa una solución completa que integra múltiples conceptos: procesamiento de XML, estructuras de datos, simulación lógica y visualización gráfica. Cada componente cumple una función específica y se integra de forma coherente dentro del sistema.

El resultado es un sistema funcional, modular y extensible, capaz de simular el comportamiento de drones para la construcción de mensajes, cumpliendo con los requerimientos establecidos y demostrando un dominio sólido de los conceptos involucrados.

El sistema cumple con los requerimientos establecidos:

1. Manejo de XML
2. Simulación de drones
3. Generación de salidas

4. Visualización gráfica
5. Gestión de entidades

POSIBLES MEJORAS

1. Persistencia de drones en XML
2. Edición de sistemas dinámicos
3. Simulación animada en tiempo real
4. Mejoras en interfaz gráfica

ANEXOS

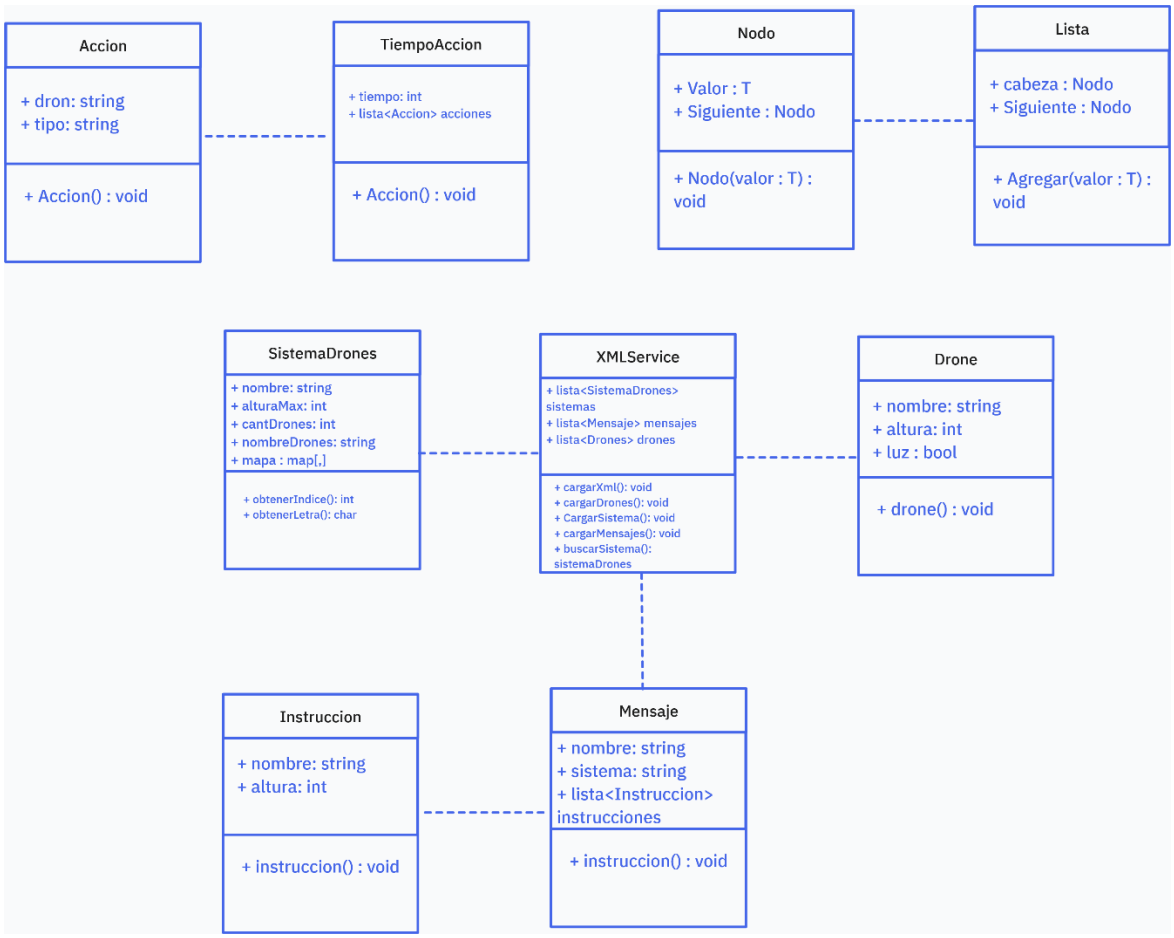


Diagrama de clases.